

Experiment – 2

Name: K. SRI LASYA

Regno: 192472216

Course Code: MLA0305

Slot: B

Title

Implementation of Reinforcement Learning Framework and Markov Decision Process (MDP)

AIM

To implement a basic **Reinforcement Learning framework** and demonstrate the components of a **Markov Decision Process (MDP)** using Python and NumPy.

PROCEDURE

1. Import the required Python libraries.
2. Define the states of the environment.
3. Define the available actions.
4. Define the transition probabilities.
5. Define the rewards associated with state transitions.
6. Define the discount factor.
7. Create an MDP environment.
8. Initialize the agent in a starting state.
9. Select actions and observe the resulting states and rewards.
10. Calculate the cumulative discounted reward.
11. Display the MDP components and agent interaction.
12. Visualize the state transitions and rewards.

MARKOV DECISION PROCESS (MDP)

Given a Markov decision process (MDP) with states S1, S2, and S3, and actions A1 and A2, with the following transition probabilities:

$$P(S2 | S1, A1) = 0.6,$$

$$P(S2 | S1, A2) = 0.4$$

$$P(S3 | S1, A1) = 0.2,$$

$$P(S3 | S1, A2) = 0.8$$

$$P(S1 | S2, A1) = 0.7,$$

$$P(S1 \mid S2, A2) = 0.3$$

$$P(S3 \mid S2, A1) = 0.5,$$

$$P(S3 \mid S2, A2) = 0.5$$

$$P(S1 \mid S3, A1) = 0.9,$$

$$P(S1 \mid S3, A2) = 0.1$$

$$P(S2 \mid S3, A1) = 0.4,$$

$$P(S2 \mid S3, A2) = 0.6$$

And rewards

$$R(S1, A1, S2) = 5,$$

$$R(S1, A2, S2) = 10,$$

$$R(S1, A1, S3) = -1,$$

$$R(S1, A2, S3) = -5,$$

$$R(S2, A1, S1) = 3,$$

$$R(S2, A2, S1) = 7,$$

$$R(S2, A1, S3) = 2,$$

$$R(S2, A2, S3) = 1,$$

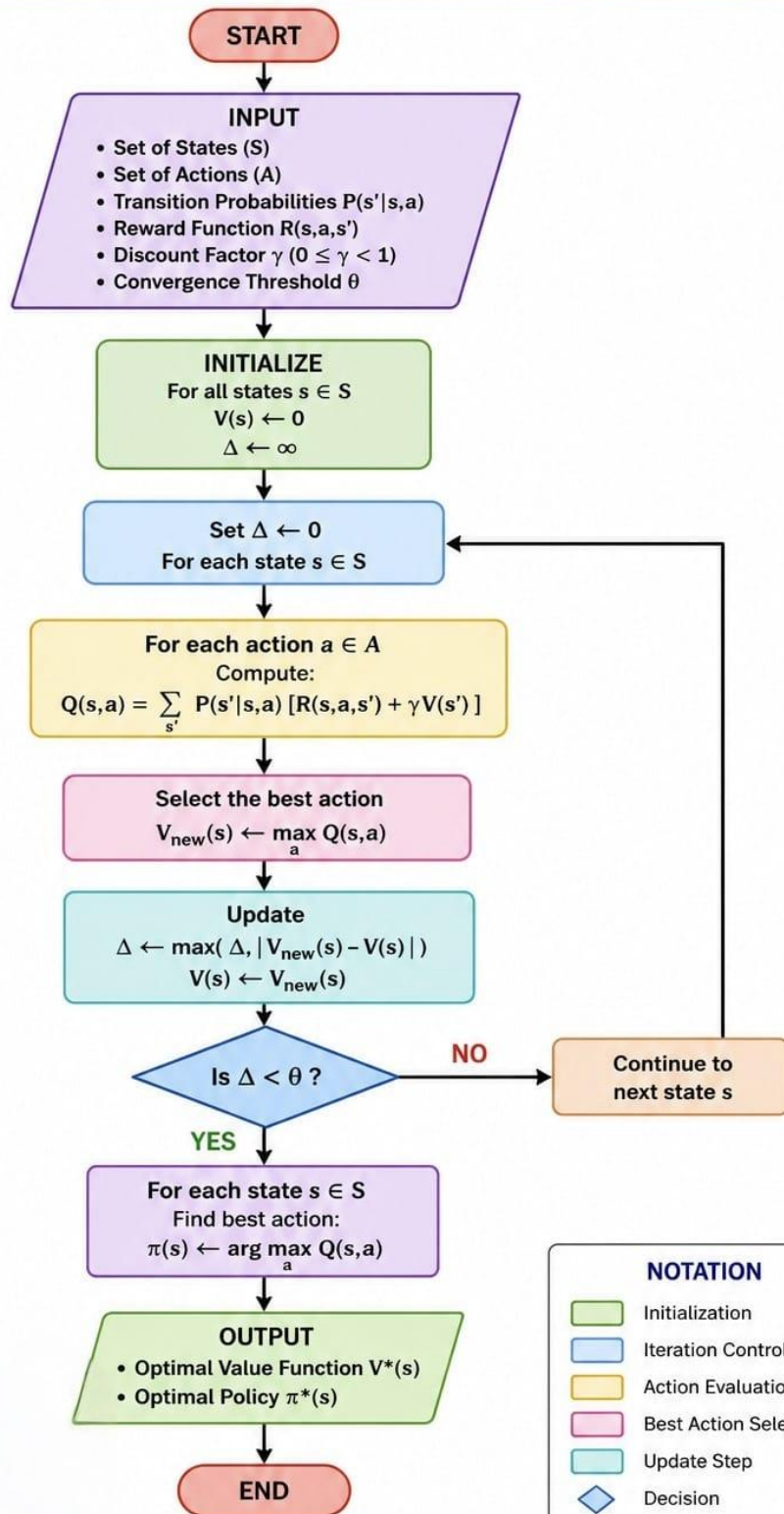
$$R(S3, A1, S1) = 4,$$

$$R(S3, A2, S1) = 6,$$

$$R(S3, A1, S2) = 0, \text{ and}$$

$$R(S3, A2, S2) = -2.$$

MARKOV DECISION PROCESS (MDP) – SOLUTION FLOWCHART



```
# MODULE 1
```

```
#####
```

```
states <- c("S1","S2","S3")
```

```
actions <- c("A1","A2")
```

```
cat("Number of States :", length(states), "\n")
```

```
cat("Number of Actions:", length(actions), "\n\n")
```

```
cat("States:\n")
```

```
print(states)
```

```
cat("\nActions:\n")
```

```
print(actions)
```

```
# Transition Probabilities
```

```
transition <- data.frame(
```

```
  CurrentState=c("S1","S1","S1","S1",  
                 "S2","S2","S2","S2",  
                 "S3","S3","S3","S3"),
```

```
  Action=c("A1","A2","A1","A2",  
           "A1","A2","A1","A2",  
           "A1","A2","A1","A2"),
```

```
  NextState=c("S2","S2","S3","S3",  
              "S1","S1","S3","S3",  
              "S1","S1","S2","S2"),
```

```
  Probability=c(0.6,0.4,0.2,0.8,  
               0.7,0.3,0.5,0.5,  
               0.9,0.1,0.4,0.6)
```

```
)
```

```
cat("\nTransition Probabilities\n")
```

```
print(transition)
```

```
# Rewards
```

```
reward <- data.frame(
```

```
  CurrentState=c("S1","S1","S1","S1",
```

```
                "S2","S2","S2","S2",
```

```
                "S3","S3","S3","S3"),
```

```
  Action=c("A1","A2","A1","A2",
```

```
           "A1","A2","A1","A2",
```

```
           "A1","A2","A1","A2"),
```

```
  NextState=c("S2","S2","S3","S3",
```

```
              "S1","S1","S3","S3",
```

```
              "S1","S1","S2","S2"),
```

```
  Reward=c(5,10,-1,-5,
```

```
           3,7,2,1,
```

```
           4,6,0,-2)
```

```
)
```

```
cat("\nReward Table\n")
```

```
print(reward)
```

```
#####
```

```
# MODULE 2
```

```
#####
```

```
A1 <- matrix(c(
```

```
0,0.6,0.2,  
0.7,0,0.5,  
0.9,0.4,0  
,3,3,byrow=TRUE)
```

```
rownames(A1) <- states  
colnames(A1) <- states
```

```
cat("\nTransition Matrix for Action A1\n")  
print(A1)
```

```
A2 <- matrix(c(  
0,0.4,0.8,  
0.3,0,0.5,  
0.1,0.6,0  
,3,3,byrow=TRUE)
```

```
rownames(A2) <- states  
colnames(A2) <- states
```

```
cat("\nTransition Matrix for Action A2\n")  
print(A2)
```

```
#####  
# MODULE 3
```

```
#####
```

```
cat("\nReward Matrix\n")  
print(reward)
```

```
#####  
# MODULE 4
```

```
#####
```

```
eir <- merge(transition,reward)
```

```
eir$ExpectedReward <- eir$Probability * eir$Reward
```

```
cat("\nExpected Reward Calculation\n")
```

```
print(eir)
```

```
summary_table <- aggregate(  
  ExpectedReward ~ CurrentState + Action,  
  data=eir,  
  sum)
```

```
cat("\nExpected Immediate Reward\n")
```

```
print(summary_table)
```

```
#####
```

```
#####
```

```
# MODULE 5
```

```
#####
```

```
cat("\nSummary Table\n")
```

```
print(summary_table)
```

```
overall_summary <- sum(summary_table$ExpectedReward)
```

```
cat("\nOverall Summary Value =", overall_summary, "\n")
```

```
# Best Action for each state
```

```
best_action <- do.call(rbind,  
  lapply(split(summary_table, summary_table$CurrentState),  
    function(x) x[which.max(x$ExpectedReward), ]))
```

```
cat("\nBest Action for Each State\n")
print(best_action)
```

```
library(igraph)
```

```
edges <- data.frame(
  from = transition$CurrentState,
  to = transition$NextState,
  label=paste0(
    transition$Action,
    "\nP=",
    transition$Probability
  )
)
```

```
g <- graph_from_data_frame(edges, directed = TRUE)
```

```
layout1 <- matrix(c(
  0,1,
  1,1,
  0.5,0
), ncol=2, byrow=TRUE)
```

```
plot(
  g,
  layout=layout1,
  vertex.color="skyblue",
  vertex.frame.color="darkblue",
  vertex.size=40,
  vertex.label.cex=1.3,
  vertex.label.color="black",
```



```

edge.color="red",
edge.width=2,
edge.curved=0.25,
edge.label=E(g)$label,
edge.label.cex=1,
edge.label.color="blue",
edge.arrow.size=0.6,
main="Markov Decision Process"
)

```

```

legend(
  "bottomleft",
  legend=c("State","Transition","Action"),
  pch=c(19,NA,NA),
  col=c("skyblue","red","blue"),
  lty=c(NA,1,NA),
  lwd=c(NA,2,NA),
  bty="n"
)

```

```

cat("\n=====\n")
cat("OPTIMAL POLICY\n")
cat("=====\n")

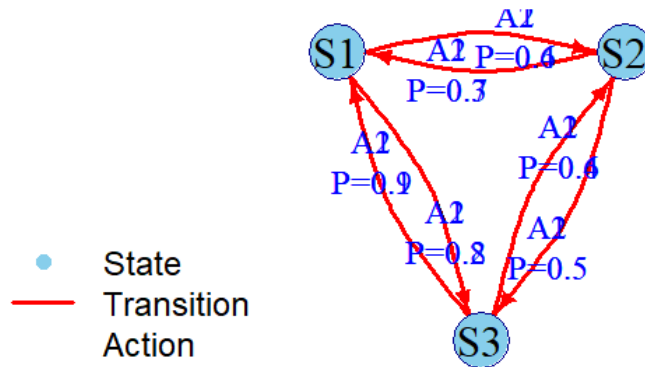
```

```

for(i in 1:nrow(best_action))
{
  cat(best_action$CurrentState[i],
    "--> ",
    best_action$Action[i],
    " (Expected Reward = ",
    best_action$ExpectedReward[i,])\n",sep="")
}

```

Markov Decision Process



```
#####
> # MARKOV DECISION PROCESS (MDP)
> #####
>
> #####
> # MODULE 1
> #####
>
> states <- c("S1","S2","S3")
> actions <- c("A1","A2")
>
> cat("Number of States :", length(states), "\n")
Number of States : 3
> cat("Number of Actions:", length(actions), "\n\n")
Number of Actions: 2
>
> cat("States:\n")
```

States:

```
> print(states)
[1] "S1" "S2" "S3"
>
> cat("\nActions:\n")
```

Actions:

```
> print(actions)
[1] "A1" "A2"
>
> # Transition Probabilities
> transition <- data.frame(
+   CurrentState=c("S1","S1","S1","S1",
+                 "S2","S2","S2","S2",
+                 "S3","S3","S3","S3"),
+   Action=c("A1","A2","A1","A2",
+            "A1","A2","A1","A2",
+            "A1","A2","A1","A2"),
+   NextState=c("S2","S2","S3","S3",
+               "S1","S1","S3","S3",
+               "S1","S1","S2","S2"),
+   Probability=c(0.6,0.4,0.2,0.8,
+                 0.7,0.3,0.5,0.5,
+                 0.9,0.1,0.4,0.6)
+ )
>
> cat("\nTransition Probabilities\n")
```

Transition Probabilities

```
> print(transition)
```

```
CurrentState Action NextState Probability
```

1	S1	A1	S2	0.6
2	S1	A2	S2	0.4
3	S1	A1	S3	0.2
4	S1	A2	S3	0.8
5	S2	A1	S1	0.7
6	S2	A2	S1	0.3
7	S2	A1	S3	0.5
8	S2	A2	S3	0.5
9	S3	A1	S1	0.9
10	S3	A2	S1	0.1
11	S3	A1	S2	0.4
12	S3	A2	S2	0.6

```
>
```

```
> # Rewards
```

```
> reward <- data.frame(
```

```
+ CurrentState=c("S1","S1","S1","S1",
```

```
+ "S2","S2","S2","S2",
```

```
+ "S3","S3","S3","S3"),
```

```
+ 
```

```
+ Action=c("A1","A2","A1","A2",
```

```
+ "A1","A2","A1","A2",
```

```
+ "A1","A2","A1","A2"),
```

```
+ 
```

```
+ NextState=c("S2","S2","S3","S3",
```

```
+ "S1","S1","S3","S3",
```

```
+ "S1","S1","S2","S2"),
```

```
+ 
```

```
+ Reward=c(5,10,-1,-5,
```

```
+ 3,7,2,1,
```

```
+ 4,6,0,-2)
```

```
+ )
>
> cat("\nReward Table\n")
```

Reward Table

```
> print(reward)
CurrentState Action NextState Reward
```

1	S1	A1	S2	5
2	S1	A2	S2	10
3	S1	A1	S3	-1
4	S1	A2	S3	-5
5	S2	A1	S1	3
6	S2	A2	S1	7
7	S2	A1	S3	2
8	S2	A2	S3	1
9	S3	A1	S1	4
10	S3	A2	S1	6
11	S3	A1	S2	0
12	S3	A2	S2	-2

```
>
> #####
> # MODULE 2
> #####
>
> A1 <- matrix(c(
+ 0,0.6,0.2,
+ 0.7,0,0.5,
+ 0.9,0.4,0
+ ),3,3,byrow=TRUE)
>
> rownames(A1) <- states
> colnames(A1) <- states
```

```
>
> cat("\nTransition Matrix for Action A1\n")
```

Transition Matrix for Action A1

```
> print(A1)
```

```
  S1 S2 S3
S1 0.0 0.6 0.2
S2 0.7 0.0 0.5
S3 0.9 0.4 0.0
```

```
>
> A2 <- matrix(c(
+ 0,0.4,0.8,
+ 0.3,0,0.5,
+ 0.1,0.6,0
+ ),3,3,byrow=TRUE)
```

```
>
> rownames(A2) <- states
> colnames(A2) <- states
```

```
>
> cat("\nTransition Matrix for Action A2\n")
```

Transition Matrix for Action A2

```
> print(A2)
```

```
  S1 S2 S3
S1 0.0 0.4 0.8
S2 0.3 0.0 0.5
S3 0.1 0.6 0.0
```

```
>
> #####
> # MODULE 3
> #####
>
```

```
> cat("\nReward Matrix\n")
```

Reward Matrix

```
> print(reward)
```

CurrentState Action NextState Reward

1	S1	A1	S2	5
2	S1	A2	S2	10
3	S1	A1	S3	-1
4	S1	A2	S3	-5
5	S2	A1	S1	3
6	S2	A2	S1	7
7	S2	A1	S3	2
8	S2	A2	S3	1
9	S3	A1	S1	4
10	S3	A2	S1	6
11	S3	A1	S2	0
12	S3	A2	S2	-2

```
>
```

```
> #####
```

```
> # MODULE 4
```

```
> #####
```

```
>
```

```
> eir <- merge(transition,reward)
```

```
>
```

```
> eir$ExpectedReward <- eir$Probability * eir$Reward
```

```
>
```

```
> cat("\nExpected Reward Calculation\n")
```

Expected Reward Calculation

```
> print(eir)
```

CurrentState Action NextState Probability Reward ExpectedReward

1	S1	A1	S2	0.6	5	3.0
---	----	----	----	-----	---	-----

2	S1	A1	S3	0.2	-1	-0.2
3	S1	A2	S2	0.4	10	4.0
4	S1	A2	S3	0.8	-5	-4.0
5	S2	A1	S1	0.7	3	2.1
6	S2	A1	S3	0.5	2	1.0
7	S2	A2	S1	0.3	7	2.1
8	S2	A2	S3	0.5	1	0.5
9	S3	A1	S1	0.9	4	3.6
10	S3	A1	S2	0.4	0	0.0
11	S3	A2	S1	0.1	6	0.6
12	S3	A2	S2	0.6	-2	-1.2

>

```
> summary_table <- aggregate(
+ ExpectedReward ~ CurrentState + Action,
+ data=eir,
+ sum)
```

>

```
> cat("\nExpected Immediate Reward\n")
```

Expected Immediate Reward

```
> print(summary_table)
```

CurrentState Action ExpectedReward

1	S1	A1	2.8
2	S2	A1	3.1
3	S3	A1	3.6
4	S1	A2	0.0
5	S2	A2	2.6
6	S3	A2	-0.6

>

```
> #####
```

```
> #####
```

```
> # MODULE 5
```



```
> #####
```

```
>
```

```
> cat("\nSummary Table\n")
```

Summary Table

```
> print(summary_table)
```

CurrentState Action ExpectedReward

1	S1	A1	2.8
---	----	----	-----

2	S2	A1	3.1
---	----	----	-----

3	S3	A1	3.6
---	----	----	-----

4	S1	A2	0.0
---	----	----	-----

5	S2	A2	2.6
---	----	----	-----

6	S3	A2	-0.6
---	----	----	------

```
>
```

```
> overall_summary <- sum(summary_table$ExpectedReward)
```

```
> cat("\nOverall Summary Value =", overall_summary, "\n")
```

Overall Summary Value = 11.5

```
>
```

```
> # Best Action for each state
```

```
> best_action <- do.call(rbind,
```

```
+      lapply(split(summary_table, summary_table$CurrentState),
```

```
+      function(x) x[which.max(x$ExpectedReward), ]))
```

```
>
```

```
> cat("\nBest Action for Each State\n")
```

Best Action for Each State

```
> print(best_action)
```

CurrentState Action ExpectedReward

S1	S1	A1	2.8
----	----	----	-----

S2	S2	A1	3.1
----	----	----	-----

S3	S3	A1	3.6
----	----	----	-----

```

>
> library(igraph)
>
> edges <- data.frame(
+   from = transition$CurrentState,
+   to = transition$NextState,
+   label=paste0(
+     transition$Action,
+     "\nP=",
+     transition$Probability
+   )
+ )
>
> g <- graph_from_data_frame(edges, directed = TRUE)
>
> layout1 <- matrix(c(
+   0,1,
+   1,1,
+   0.5,0
+ ), ncol=2, byrow=TRUE)
>
> plot(
+   g,
+   layout=layout1,
+   vertex.color="skyblue",
+   vertex.frame.color="darkblue",
+   vertex.size=40,
+   vertex.label.cex=1.3,
+   vertex.label.color="black",
+   edge.color="red",
+   edge.width=2,
+   edge.curved=0.25,

```

```

+ edge.label=E(g)$label,
+ edge.label.cex=1,
+ edge.label.color="blue",
+ edge.arrow.size=0.6,
+ main="Markov Decision Process"
+ )
>
> legend(
+ "bottomleft",
+ legend=c("State","Transition","Action"),
+ pch=c(19,NA,NA),
+ col=c("skyblue","red","blue"),
+ lty=c(NA,1,NA),
+ lwd=c(NA,2,NA),
+ bty="n"
+ )
>
> cat("\n===== \n")

=====

> cat("OPTIMAL POLICY\n")
OPTIMAL POLICY
> cat("===== \n")

=====

>
> for(i in 1:nrow(best_action))
+ {
+   cat(best_action$CurrentState[i],
+     " --> ",
+     best_action$Action[i],
+     " (Expected Reward = ",
+     best_action$ExpectedReward[i],")\n",sep="")

```

+ }

S1 --> A1 (Expected Reward = 2.8)

S2 --> A1 (Expected Reward = 3.1)

S3 --> A1 (Expected Reward = 3.6)

RESULT

The Reinforcement Learning framework and **Markov Decision Process (MDP)** were successfully implemented using Python and NumPy. The agent interacted with the environment by selecting actions, transitioning between states, receiving rewards, and calculating the discounted return. The MDP state rewards were successfully visualized using a bar graph.